

```

"""
polaritron_mc290_echo_circuit.py
Aural15 – Cirq Orchestrator
DCGP.AI / DocGrokHealth

Modular Polyhedral Echo Circuit v2
Echo Operator Lossless Reconstruction Protocol (EOLRP) – Cirq Implementation
Governance metadata, telemetry hooks, Saturn Ring 4 audit logging,
DocGrokHealth data layer output.

Patent Pending – Joshua Lopez, DCGP.AI – June 8, 2026
"""

import cirq
import numpy as np
import json
from datetime import datetime, timezone
from typing import Optional

# _____
# GOVERNANCE METADATA SCHEMA
# _____

def governance_tag(
    layer_id: int,
    phase_index: int,
    echo_state: str,
    obligation_resource: float,
    Q_L_conservation_flag: bool
) -> dict:
    """
    Governance metadata tag – attached to each circuit stage.
    Compatible with Saturn Ring 4 (Pluto memory write target).
    """
    return {
        "layer_id": layer_id,
        "phase_index": phase_index,
        "echo_state": echo_state,
        "obligation_resource": round(obligation_resource, 6),
        "Q_L_conservation_flag": Q_L_conservation_flag,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "circuit_id": "POLARITRON_MC290_ECHO_V2",
        "governance_layer": "AURAL-M/OL",
    }

```

```

        "saturn_ring": 4,
        "write_target": "PLUTO"
    }

# -----
# AUDIT LOG – Saturn Ring 4 compatible
# -----

class GovernanceAuditLog:
    """
    Immutable append-only audit log for circuit execution events.
    Feeds Saturn Ring 4 / Pluto memory write target.
    """
    def __init__(self):
        self._entries = []

    def log(self, event_type: str, metadata: dict, snapshot: Optional[dict] = None) -> dict:
        entry = {
            "event_type": event_type,
            "metadata": metadata,
            "snapshot": snapshot or {},
            "entry_index": len(self._entries),
            "logged_at": datetime.now(timezone.utc).isoformat()
        }
        self._entries.append(entry)
        return entry

    def export(self) -> list:
        return list(self._entries)

    def export_json(self, indent: int = 2) -> str:
        return json.dumps(self._entries, indent=indent)

# -----
# CORE CIRCUIT BUILDER
# -----

def build_polyhedral_echo_circuit(
    n_qubits: int = 8,
    n_layers: int = 4,
    audit_log: Optional[GovernanceAuditLog] = None
) -> cirq.Circuit:
    """
    Build the modular polyhedral echo circuit.

```

Args:

```
n_qubits: Number of qubits (8 default, scalable to 100)
n_layers: Number of entanglement + phase accumulation layers
audit_log: GovernanceAuditLog instance for Saturn Ring 4 logging
```

Returns:

```
cirq.Circuit — fully constructed echo circuit with governance metadata
"""
```

```
if audit_log is None:
```

```
    audit_log = GovernanceAuditLog()
```

```
qubits = cirq.LineQubit.range(n_qubits)
```

```
circuit = cirq.Circuit()
```

```
# — STAGE 0: INITIALIZATION —————
```

```
# Hadamard superposition + Rz phase diversity
```

```
# Obligation at initialization = 1.0 (full resource available)
```

```
obligation = 1.0
```

```
init_ops = []
```

```
for i in range(n_qubits):
```

```
    init_ops.append(cirq.H(qubits[i]))
```

```
    init_ops.append(cirq.Rz(rads=np.pi * (i + 1) / n_qubits).on(qubits[i]))
```

```
circuit.append(init_ops)
```

```
tag = governance_tag(
```

```
    layer_id=0,
```

```
    phase_index=0,
```

```
    echo_state="INITIALIZED",
```

```
    obligation_resource=obligation,
```

```
    Q_L_conservation_flag=True
```

```
)
```

```
audit_log.log(
```

```
    event_type="STAGE_INIT",
```

```
    metadata=tag,
```

```
    snapshot={"n_qubits": n_qubits, "phase_diversity": "Rz_pi_scaled"}  
)
```

```
# — STAGES 1..N: CROSS-LAYER ENTANGLEMENT + PHASE ACCUMULATION —
```

```
# Each layer: CNOT chain (sequential entanglement) + Rx rotation (phase accum
```

```
# Obligation decays slightly per layer — models resource consumption
```

```
# Q_L conservation checked per layer
```

```
for layer in range(n_layers):
```

```
    layer_ops = []
```

```

# CNOT entanglement chain
for i in range(n_qubits - 1):
    layer_ops.append(cirq.CNOT(qubits[i], qubits[i + 1]))

circuit.append(layer_ops)

# Rx phase accumulation — scales with layer depth
rx_moment = cirq.Moment([
    cirq.Rx(rads=np.pi / 4 * (layer + 1)).on(q) for q in qubits
])
circuit.append(rx_moment)

# Obligation decays per layer (models  $O = \alpha(O)$  scaling)
obligation = max(0.0, obligation - (1.0 / (n_layers + 1)))

# Q_L conservation: True while obligation >  $O_{\min}$  threshold
 $O_{\min} = 0.1$ 
q_l_flag = obligation >=  $O_{\min}$ 

tag = governance_tag(
    layer_id=layer + 1,
    phase_index=layer,
    echo_state="ENTANGLED_ACCUMULATING",
    obligation_resource=obligation,
    Q_L_conservation_flag=q_l_flag
)
audit_log.log(
    event_type="STAGE_LAYER",
    metadata=tag,
    snapshot={
        "layer": layer + 1,
        "cnot_chain_length": n_qubits - 1,
        "rx_angle_rad": round(np.pi / 4 * (layer + 1), 6),
        "obligation_remaining": round(obligation, 6)
    }
)

# — STAGE FINAL: ECHO RETURN —————
# Global Hadamard — refocuses all phases
# Echo Operator convergence to  $S^*$  — governing invariant re-instantiated

echo_return = cirq.Moment([cirq.H(qubits[i]) for i in range(n_qubits)])
circuit.append(echo_return)

tag = governance_tag(
    layer_id=n_layers + 1,
    phase_index=n_layers,

```

```

        echo_state="ECHO_RETURN",
        obligation_resource=obligation,
        Q_L_conservation_flag=obligation >= O_min
    )
    audit_log.log(
        event_type="STAGE_ECHO_RETURN",
        metadata=tag,
        snapshot={"echo_operator": "H_global", "convergence": "S*_re-instantiated"
    )

# — MEASUREMENT —————
circuit.append(cirq.measure(*qubits, key='polaritron_echo'))

audit_log.log(
    event_type="MEASUREMENT",
    metadata=governance_tag(
        layer_id=n_layers + 2,
        phase_index=n_layers + 1,
        echo_state="MEASURED",
        obligation_resource=obligation,
        Q_L_conservation_flag=obligation >= O_min
    ),
    snapshot={"measurement_key": "polaritron_echo", "n_qubits_measured": n_qu
)

return circuit

# —————
# SIMULATOR + DOCGROKHEALTH OUTPUT
# —————

def run_echo_circuit(
    n_qubits: int = 8,
    n_layers: int = 4,
    repetitions: int = 1000
) -> dict:
    """
    Run the polyhedral echo circuit and return DocGrokHealth-formatted output.

    Returns structured JSON-compatible dict:
    {
        "echo_response":      raw measurement counts,
        "fidelity_estimate":  float — estimated fidelity from dominant bitstring,
        "anomaly_flag":       bool — True if fidelity below threshold,
        "governance_audit":   full Saturn Ring 4 audit log,
        "circuit_summary":    human-readable circuit description
    }

```

```

}
"""
audit_log = GovernanceAuditLog()

circuit = build_polyhedral_echo_circuit(
    n_qubits=n_qubits,
    n_layers=n_layers,
    audit_log=audit_log
)

simulator = cirq.Simulator()
result = simulator.run(circuit, repetitions=repetitions)
counts = result.multi_measurement_histogram(keys=['polaritron_echo'])

# Fidelity estimate - dominant bitstring probability
total = sum(counts.values())
dominant_count = max(counts.values()) if counts else 0
fidelity_estimate = dominant_count / total if total > 0 else 0.0

# Anomaly flag - fidelity below governed threshold
FIDELITY_THRESHOLD = 0.05 # Expected near-uniform for echo circuit
anomaly_flag = fidelity_estimate > (1.0 / (2 ** n_qubits)) * 10

# DocGrokHealth output
output = {
    "echo_response": {
        str(k): int(v) for k, v in sorted(
            counts.items(), key=lambda x: -x[1]
        )[:10] # Top 10 bitstrings
    },
    "fidelity_estimate": round(fidelity_estimate, 6),
    "anomaly_flag": anomaly_flag,
    "circuit_metadata": {
        "n_qubits": n_qubits,
        "n_layers": n_layers,
        "repetitions": repetitions,
        "circuit_id": "POLARITRON_MC290_ECHO_V2",
        "docgrok_lane": "DocGrokHealth",
        "saturn_ring_target": 4,
        "pluto_write": True,
        "governance_framework": "AURAL-M/EOLRP",
        "patent_pending": "Joshua Lopez, DCGP.AI, June 8, 2026"
    },
    "governance_audit": audit_log.export(),
    "circuit_summary": str(circuit)
}

```

```

    return output

# -----
# MAIN
# -----

if __name__ == "__main__":
    print("=" * 60)
    print("POLARITRON MC290 – POLYHEDRAL ECHO CIRCUIT V2")
    print("DCGP.AI / AURA115 / DocGrokHealth")
    print("Patent Pending – Joshua Lopez – June 8, 2026")
    print("=" * 60)

    # Build and display circuit
    audit = GovernanceAuditLog()
    circuit = build_polyhedral_echo_circuit(n_qubits=8, n_layers=4, audit_log=audit)
    print("\nCIRCUIT:")
    print(circuit)

    # Run and display DocGrokHealth output
    print("\nRUNNING (1000 repetitions)...")
    output = run_echo_circuit(n_qubits=8, n_layers=4, repetitions=1000)

    print(f"\nFIDELITY ESTIMATE: {output['fidelity_estimate']}")
    print(f"ANOMALY FLAG: {output['anomaly_flag']}")
    print(f"\nTOP ECHO RESPONSES:")
    for bitstring, count in list(output['echo_response'].items())[:5]:
        print(f"  {bitstring}: {count}")

    print(f"\nGOVERNANCE AUDIT ({len(output['governance_audit'])} entries logged)
    print("\nFull output available via run_echo_circuit() – JSON-ready for DocGro

```